

Rust

Violet

2026-04-20

Chapter 0

目 录

I	JS & TS 核心基础	
1.	JS & TS 概述与环境搭建	5
2.	基础语法与变量	6
3.	函数与作用域	7
4.	对象与数组	8
II	JavaScript 核心特性	
5.	原型与面向对象	10
6.	this 与执行上下文	11
7.	异步编程与事件循环	12
7.1.	JavaScript 异步模型概览	12
7.2.	事件循环机制	12
7.3.	Promise: 异步编程的核心	12
7.4.	async/await: 语法糖的威力	23
7.5.	高级异步模式	33
7.6.	定时器与异步调度	44
7.7.	实战示例	44



JS & TS 核心基础

1. JS & TS 概述与环境搭建 ·····	5
2. 基础语法与变量 ·····	6
3. 函数与作用域 ·····	7
4. 对象与数组 ·····	8

Chapter 1

JS & TS 概述与环境搭建

Chapter 2

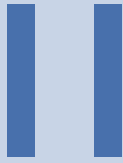
基础语法与变量

Chapter 3

函数与作用域

Chapter 4

对象与数组



JavaScript 核心特性

5. 原型与面向对象·····	10
6. <code>this</code> 与执行上下文·····	11
7. 异步编程与事件循环·····	12

Chapter 5

原型与面向对象

Chapter 6

this 与执行上下文

Chapter 7

异步编程与事件循环

1 JavaScript 异步模型概览

1.1 同步 vs 异步

1.2 为什么需要异步

1.3 异步编程的演进历程

2 事件循环机制

2.1 调用栈 (Call Stack)

2.2 任务队列 (Task Queue)

2.3 微任务与宏任务

2.4 事件循环执行流程

2.5 浏览器 vs Node.js 事件循环

3 Promise：异步编程的核心

Promise 是 JavaScript 异步编程的核心抽象，代表一个异步操作的最终完成（或失败）及其结果值。

3.1 Promise 基础概念

Promise 是一个对象，用于表示异步操作的最终完成（或失败）。

三种状态：

状态	说明	可否改变
pending（等待中）	初始状态，既不是成功也不是失败	✓

fulfilled (已成功)	操作成功完成	X
rejected (已失败)	操作失败	X

状态转换：

```
1 pending → fulfilled (resolve)
2 pending → rejected (reject)
```

基本语法：

```
1 const promise = new Promise((resolve, reject) => {
2   // 异步操作
3   setTimeout(() => {
4     const success = true;
5
6     if (success) {
7       resolve("操作成功"); // 状态变为 fulfilled
8     } else {
9       reject(new Error("操作失败")); // 状态变为 rejected
10    }
11  }, 1000);
12 });
13
14 promise
15 .then(result => {
16   console.log(result); // "操作成功"
17 })
18 .catch(error => {
19   console.error(error); // 如果失败则执行
20 });
```

Promise 的特点：

特点	说明
状态不可逆	一旦从 pending 变为 fulfilled/rejected，状态不再改变
链式调用	then/catch 返回新的 Promise，可以链式调用
微任务	Promise 回调作为微任务执行，优先级高于宏任务
错误冒泡	错误会沿着链式调用向下传递，直到被 catch 捕获
立即执行	Promise 构造函数中的代码会立即执行

3.2 Promise 状态机

Promise 的状态变化遵循严格的状态机规则。

状态转换图：

```
1      resolve()
2 pending -----> fulfilled
3   |               |
4   |               | then() 回调
```



示例：状态变化：

```

1  // 示例 1: 成功状态
2  const p1 = new Promise((resolve, reject) => {
3    console.log("Promise 创建"); // 立即执行
4    resolve("成功");
5  });
6
7  p1.then(value => {
8    console.log(value); // "成功"
9    return "新值";
10 }).then(value => {
11   console.log(value); // "新值"
12 });
13
14 // 示例 2: 失败状态
15 const p2 = new Promise((resolve, reject) => {
16   reject(new Error("失败"));
17 });
18
19 p2.catch(error => {
20   console.error(error.message); // "失败"
21 });
22
23 // 示例 3: 状态不可逆
24 const p3 = new Promise((resolve, reject) => {
25   resolve("第一次");
26   reject("第二次"); // 这行不会生效
27 });
28
29 p3.then(value => {
30   console.log(value); // "第一次"
31 });

```

Pending 状态的持久性：

```

1  // Promise 一直处于 pending 状态
2  const pendingPromise = new Promise(() => {
3    // 既不 resolve 也不 reject
4  });
5
6  pendingPromise.then(() => {
7    console.log("永远不会执行");
8  });

```

```
8  });
9
10 // 超时控制
11 const timeoutPromise = new Promise((_, reject) => {
12   setTimeout(() => {
13     reject(new Error("Timeout"));
14   }, 5000);
15 });
```

3.3 then/catch/finally

这三个方法是 Promise 的核心 API。

then 方法：

`then` 方法接受两个可选参数：成功回调和失败回调。

```
1 promise.then(
2   onFulfilled, // 可选, 当 Promise fulfilled 时调用
3   onRejected   // 可选, 当 Promise rejected 时调用
4 );
```

 JavaScript

基本用法：

```
1 const promise = fetchData();
2
3 // 方式 1: 两个参数
4 promise.then(
5   result => console.log("成功:", result),
6   error  => console.error("失败:", error)
7 );
8
9 // 方式 2: then + catch (推荐)
10 promise
11   .then(result => console.log("成功:", result))
12   .catch(error => console.error("失败:", error));
```

 JavaScript

返回值：

```
1 // then 返回新的 Promise
2 promise
3   .then(result => {
4     console.log(result); // "原始值"
5     return "新值"; // 返回新值, 下一个 then 接收
6   })
7   .then(newResult => {
8     console.log(newResult); // "新值"
9     return Promise.resolve("Promise 值"); // 返回 Promise
10  })
11   .then(finalResult => {
12     console.log(finalResult); // "Promise 值"
13   });
```

 JavaScript

catch 方法：

`catch` 是 `then(null, onRejected)` 的语法糖。

```
1 // 这两种写法等价
2 promise.catch(error => console.error(error));
3 promise.then(null, error => console.error(error));
```

 JavaScript

错误捕获：

```
1 fetchData()
2   .then(result => {
3     // 如果这里抛出错误，会被后面的 catch 捕获
4     if (!result) {
5       throw new Error("数据为空");
6     }
7     return process(result);
8   })
9   .then(processed => {
10    return save(processed);
11  })
12  .catch(error => {
13    // 捕获上面任何一步的错误
14    console.error("错误:", error.message);
15  });
```

 JavaScript

NOTE

`catch` 会捕获前面所有步骤的错误，包括 `then` 回调中抛出的错误。

`finally` 方法：

`finally` 无论 Promise 成功还是失败都会执行，常用于清理工作。

```
1 showLoading();
2
3 fetchData()
4   .then(result => {
5     display(result);
6   })
7   .catch(error => {
8     showError(error);
9   })
10  .finally(() => {
11    // 无论成功还是失败，都会执行
12    hideLoading();
13  });
```

 JavaScript

特点：

特点	说明
必定执行	无论 fulfilled 还是 rejected 都会执行
无参数	不接收任何参数
不改变结果	返回值不影响 Promise 的最终结果
透传	继续传递原始的 resolved/rejected 值

示例:

```
1 Promise.resolve("成功")
2   .finally(() => {
3     console.log("清理工作");
4     return "被忽略"; // 这个返回值会被忽略
5   })
6   .then(value => {
7     console.log(value); // "成功" (原始值)
8   });
9
10 Promise.reject(new Error("失败"))
11   .finally(() => {
12     console.log("清理工作");
13   })
14   .catch(error => {
15     console.error(error.message); // "失败" (原始错误)
16   });
```

 JavaScript

3.4 Promise 链式调用

Promise 的强大之处在于链式调用，可以避免回调地狱。

基本链式调用:

```
1 // 串行执行: 每一步依赖上一步的结果
2 fetchUser(userId)
3   .then(user => {
4     return fetchPosts(user.id); // 返回 Promise
5   })
6   .then(posts => {
7     return fetchComments(posts[0].id); // 返回 Promise
8   })
9   .then(comments => {
10    console.log(comments);
11  })
12  .catch(error => {
13    console.error("任何一步出错都会到这里", error);
14  });
```

 JavaScript

返回值类型:

```
1 promise
2   .then(result => {
3     // 情况 1: 返回普通值
4     return "字符串";
5     // 下一个 then 接收: "字符串"
6   })
7   .then(value => {
8     // 情况 2: 返回 Promise
9     return Promise.resolve(42);
10    // 下一个 then 接收: 42
11  })
```

 JavaScript

```
12 .then(value => {
13     // 情况 3: 不返回 (返回 undefined)
14     console.log(value); // 42
15     // 下一个 then 接收: undefined
16 })
17 .then(value => {
18     console.log(value); // undefined
19 });
```

链式调用的错误传播:

```
1 step1()
2 .then(result1 => step2(result1))
3 .then(result2 => step3(result2)) // 如果 step2 失败, 跳过这里
4 .then(result3 => step4(result3)) // 如果 step3 失败, 跳过这里
5 .catch(error => {
6     // 捕获 step1、step2、step3、step4 中任何一个的错误
7     console.error(error);
8 });
```

 JavaScript

局部错误处理:

```
1 step1()
2 .then(result1 => {
3     return step2(result1).catch(error => {
4         // 只捕获 step2 的错误
5         console.warn("step2 失败, 使用默认值");
6         return defaultValue;
7     });
8 })
9 .then(result2 => {
10     // 即使 step2 失败, 这里仍会执行 (使用默认值)
11     return step3(result2);
12 })
13 .catch(error => {
14     // 捕获 step1 或 step3 的错误
15     console.error(error);
16 });
```

 JavaScript

3.5 Promise 静态方法

Promise 提供了多个静态方法来处理多个 Promise。

Promise.all:

等待所有 Promise 都成功, 返回结果数组; 如果有一个失败, 立即拒绝。

```
1 const promise1 = Promise.resolve(3);
2 const promise2 = 42; // 非 Promise 值会被包装
3 const promise3 = new Promise(resolve => {
4     setTimeout(resolve, 100, "foo");
5 });
6
```

 JavaScript

```
7 Promise.all([promise1, promise2, promise3])
8   .then(values => {
9     console.log(values); // [3, 42, "foo"]
10  })
11  .catch(error => {
12    // 如果任何一个 Promise reject, 立即执行
13    console.error(error);
14  });
```

失败快速:

```
1 const p1 = Promise.resolve(1);
2 const p2 = Promise.reject(new Error("失败"));
3 const p3 = Promise.resolve(3);
4
5 Promise.all([p1, p2, p3])
6   .then(values => {
7     console.log(values); // 不会执行
8   })
9   .catch(error => {
10    console.error(error.message); // "失败"
11    // 注意: p3 可能还在执行, 但结果被忽略
12  });
```

 JavaScript

应用场景:

```
1 // 并行加载多个资源
2 async function loadResources() {
3   const [users, posts, comments] = await Promise.all([
4     fetch("/api/users").then(r => r.json()),
5     fetch("/api/posts").then(r => r.json()),
6     fetch("/api/comments").then(r => r.json())
7   ]);
8
9   return { users, posts, comments };
10 }
```

 JavaScript

Promise.race:

返回第一个 settled (fulfilled 或 rejected) 的 Promise 的结果。

```
1 const p1 = new Promise(resolve => {
2   setTimeout(resolve, 500, "first");
3 });
4
5 const p2 = new Promise(resolve => {
6   setTimeout(resolve, 100, "second");
7 });
8
9 Promise.race([p1, p2])
10  .then(value => {
11    console.log(value); // "second" (更快的那个)
12  });
```

 JavaScript

超时控制:

```
1 function fetchWithTimeout(url, timeout) {⚡ JavaScript  
2   const fetchPromise = fetch(url);  
3   const timeoutPromise = new Promise( (_, reject) => {  
4     setTimeout(() => reject(new Error("Request timeout")), timeout);  
5   });  
6  
7   return Promise.race([fetchPromise, timeoutPromise]);  
8 }  
9  
10 // 使用  
11 fetchWithTimeout("https://api.example.com/data", 5000)  
12   .then(response => response.json())  
13   .catch(error => {  
14     console.error(error.message); // "Request timeout" 或网络错误  
15   });
```

Promise.allSettled:

等待所有 Promise 都 settled，返回每个 Promise 的状态和结果。

```
1 const p1 = Promise.resolve(1);⚡ JavaScript  
2 const p2 = Promise.reject(new Error("失败"));  
3 const p3 = Promise.resolve(3);  
4  
5 Promise.allSettled([p1, p2, p3])  
6   .then(results => {  
7     console.log(results);  
8     // [  
9     //   { status: "fulfilled", value: 1 },  
10    //   { status: "rejected", reason: Error: 失败 },  
11    //   { status: "fulfilled", value: 3 }  
12    // ]  
13  });
```

处理部分失败:

```
1 async function loadAllResources(urls) {⚡ JavaScript  
2   const promises = urls.map(url =>  
3     fetch(url).then(r => r.json()).catch(error => ({ error })))  
4   );  
5  
6   const results = await Promise.allSettled(promises);  
7  
8   const successful = results  
9     .filter(r => r.status === "fulfilled")  
10    .map(r => r.value);  
11  
12   const failed = results  
13     .filter(r => r.status === "rejected")  
14     .map(r => r.reason);  
15  
16   console.log("成功:", successful.length);  
17   console.log("失败:", failed.length);  
18 }
```

```
19   return { successful, failed };
20 }
```

Promise.any:

返回第一个 fulfilled 的 Promise 的结果；如果全部 rejected，返回 AggregateError。

```
1  const p1 = Promise.reject(new Error("失败 1"));
2  const p2 = Promise.resolve("成功");
3  const p3 = Promise.reject(new Error("失败 2"));
4
5  Promise.any([p1, p2, p3])
6    .then(value => {
7      console.log(value); // "成功"
8    })
9    .catch(error => {
10     // 只有全部失败才会到这里
11     console.error(error); // AggregateError
12   });
```

 JavaScript

全部失败:

```
1  const p1 = Promise.reject(new Error("失败 1"));
2  const p2 = Promise.reject(new Error("失败 2"));
3
4  Promise.any([p1, p2])
5    .then(value => {
6      console.log(value); // 不会执行
7    })
8    .catch(error => {
9      console.error(error.name); // "AggregateError"
10     console.error(error.errors); // [Error: 失败 1, Error: 失败 2]
11   });
```

 JavaScript

应用场景：多源数据获取：

```
1  // 从多个 CDN 获取数据，使用最快的成功响应
2  const cdns = [
3    fetch("https://cdn1.example.com/data.json"),
4    fetch("https://cdn2.example.com/data.json"),
5    fetch("https://cdn3.example.com/data.json")
6  ];
7
8  Promise.any(cdns)
9    .then(response => response.json())
10   .then(data => {
11     console.log("从最快的 CDN 获取数据:", data);
12   })
13   .catch(error => {
14     console.error("所有 CDN 都失败:", error.errors);
15   });
```

 JavaScript

对比总结：

方法	成功条件	失败条件	返回值
Promise.all	全部 fulfilled	任一 rejected	结果数组 / 第一个错误
Promise.race	第一个 settled	第一个 rejected	第一个结果 / 错误
Promise.allSettled	全部 settled	永不失败	状态数组
Promise.any	任一 fulfilled	全部 rejected	第一个成功 / AggregateError

3.6 错误处理最佳实践

良好的错误处理是 Promise 使用的关键。

始终添加 catch:

```
1 // ❌ 错误做法: 未捕获的错误
2 fetchData().then(result => {
3   console.log(result);
4 });
5
6 // ✅ 正确做法: 总是添加 catch
7 fetchData()
8   .then(result => {
9     console.log(result);
10  })
11  .catch(error => {
12    console.error("Error:", error);
13  });
```

JavaScript

避免吞掉错误:

```
1 // ❌ 错误做法: 吞掉错误
2 promise.catch(error => {
3   // 什么都不做
4 });
5
6 // ✅ 正确做法: 记录或重新抛出
7 promise.catch(error => {
8   console.error(error);
9   // 或者重新抛出, 让上层处理
10  throw error;
11 });
```

JavaScript

统一的错误处理:

```
1 // 全局未处理的 Promise rejection
2 window.addEventListener("unhandledrejection", event => {
3   console.error("Unhandled rejection:", event.reason);
4   event.preventDefault(); // 防止控制台警告
5 });
6
7 // Node.js
8 process.on("unhandledRejection", (reason, promise) => {
9   console.error("Unhandled Rejection at:", promise, "reason:", reason);
```

JavaScript

```
10 });
```

async/await 中的错误处理:

```
1 // 方式 1: try-catch
2 async function loadData() {
3   try {
4     const response = await fetch("/api/data");
5     const data = await response.json();
6     return data;
7   } catch (error) {
8     console.error("Failed to load data:", error);
9     throw error; // 重新抛出或返回默认值
10  }
11 }
12
13 // 方式 2: .catch()
14 async function loadData() {
15   return fetch("/api/data")
16     .then(response => response.json())
17     .catch(error => {
18       console.error("Failed to load data:", error);
19       return null; // 返回默认值
20     });
21 }
```

4 async/await: 语法糖的威力

`async/await` 是基于 Promise 的语法糖, 让异步代码看起来像同步代码, 提高可读性和可维护性。

4.1 基本语法

`async` 关键字声明一个异步函数, `await` 关键字等待 Promise 完成。

| async 函数

```
1 // 声明 async 函数
2 async function fetchData() {
3   return "data";
4 }
5
6 // async 函数总是返回 Promise
7 fetchData().then(data => {
8   console.log(data); // "data"
9 });
10
11 // 等价于
12 function fetchData() {
13   return Promise.resolve("data");
14 }
```

特点:

特点	说明
返回值	自动包装为 Promise
非 async 函数调用	可以调用 async 函数
异常处理	抛出的异常会被 reject
兼容性	ES2017+

await 表达式

`await` 只能在 `async` 函数内部使用，用于等待 Promise 完成。

```
1  async function loadData() {
2    // 等待 Promise 完成
3    const response = await fetch("/api/data");
4
5    // 继续等待 JSON 解析
6    const data = await response.json();
7
8    return data;
9  }
10
11 // 使用
12 loadData().then(data => {
13   console.log(data);
14 });
```

JavaScript

执行流程:

```
1  async function example() {
2    console.log("1. 开始");
3
4    const result = await someAsyncOperation();
5    console.log("2. 结果:", result);
6
7    console.log("3. 结束");
8  }
9
10 // 输出顺序:
11 // 1. 开始
12 // (等待 someAsyncOperation 完成)
13 // 2. 结果: xxx
14 // 3. 结束
```

JavaScript

对比: Promise vs async/await

```
1  // Promise 链式调用
2  function loadUserData(userId) {
3    return fetchUser(userId)
4      .then(user => fetchPosts(user.id))
5      .then(posts => fetchComments(posts[0].id))
```

JavaScript


```
6     .then(comments => {
7         return { user, posts, comments };
8     })
9     .catch(error => {
10         console.error(error);
11     });
12 }
13
14 // async/await (更清晰)
15 async function loadUserData(userId) {
16     try {
17         const user = await fetchUser(userId);
18         const posts = await fetchPosts(user.id);
19         const comments = await fetchComments(posts[0].id);
20
21         return { user, posts, comments };
22     } catch (error) {
23         console.error(error);
24     }
25 }
```

4.2 错误处理

async/await 提供了更直观的错误处理方式。

try-catch 方式

```
1  async function loadData() {
2      try {
3          const response = await fetch("/api/data");
4
5          if (!response.ok) {
6              throw new Error(`HTTP error! status: ${response.status}`);
7          }
8
9          const data = await response.json();
10         return data;
11
12     } catch (error) {
13         console.error("Failed to load data:", error.message);
14
15         // 可以选择:
16         // 1. 重新抛出
17         throw error;
18
19         // 2. 返回默认值
20         // return defaultValue;
21
22         // 3. 返回 null
23         // return null;
24     }
25 }
```

多个 await 的错误处理

```
1  async function loadAllData() {
2      try {
3          const users = await fetchUsers();
4          const posts = await fetchPosts();
5          const comments = await fetchComments();
6
7          return { users, posts, comments };
8
9      } catch (error) {
10         // 任何一个 await 失败都会到这里
11         console.error("Failed to load data:", error);
12         throw error;
13     }
14 }
```

NOTE

如果第一个 await 失败，后面的 await 不会执行。

独立的错误处理

```
1  async function loadAllData() {
2      let users, posts, comments;
3
4      // 每个请求独立处理错误
5      try {
6          users = await fetchUsers();
7      } catch (error) {
8          console.warn("Failed to load users:", error);
9          users = [];
10     }
11
12     try {
13         posts = await fetchPosts();
14     } catch (error) {
15         console.warn("Failed to load posts:", error);
16         posts = [];
17     }
18
19     try {
20         comments = await fetchComments();
21     } catch (error) {
22         console.warn("Failed to load comments:", error);
23         comments = [];
24     }
25
26     return { users, posts, comments };
27 }
```

Promise.catch 混合使用

```
1  async function loadData() {
```

```
2 // 方式 1: await + try-catch
3 try {
4   const data = await fetchData();
5   return data;
6 } catch (error) {
7   console.error(error);
8 }
9
10 // 方式 2: await + .catch()
11 const data = await fetchData().catch(error => {
12   console.error(error);
13   return null;
14 });
15
16 return data;
17 }
```

4.3 并行执行 vs 串行执行

理解并行和串行的区别对于性能优化至关重要。

串行执行（不推荐）

```
1 // ❌ 串行执行: 总耗时 = 1s + 1s + 1s = 3s
2 async function loadSerially() {
3   const users = await fetchUsers(); // 1s
4   const posts = await fetchPosts(); // 1s
5   const comments = await fetchComments(); // 1s
6
7   return { users, posts, comments };
8 }
```

 JavaScript

执行时间线:

```
1 0s -----> 1s -----> 2s -----> 3s
2 |fetchUsers|fetchPosts|fetchComments|
```

并行执行（推荐）

```
1 // ✅ 并行执行: 总耗时 = max(1s, 1s, 1s) = 1s
2 async function loadInParallel() {
3   const [users, posts, comments] = await Promise.all([
4     fetchUsers(),
5     fetchPosts(),
6     fetchComments()
7   ]);
8
9   return { users, posts, comments };
10 }
```

 JavaScript

执行时间线:

```
1 0s -----> 1s
```

```
2 | fetchUsers      |
3 | fetchPosts      |
4 | fetchComments    |
```

部分并行

当某些请求依赖其他请求的结果时，可以部分并行。

```
1  async function loadUserData(userId) {
2    // 第一步：获取用户（必须串行）
3    const user = await fetchUser(userId);
4
5    // 第二步：并行获取用户的帖子和评论
6    const [posts, comments] = await Promise.all([
7      fetchPosts(user.id),
8      fetchComments(user.id)
9    ]);
10
11   return { user, posts, comments };
12 }
```

执行时间线：

```
1  0s -----> 1s -----> 2s
2  | fetchUser | fetchPosts      |
3  |           | fetchComments   |
```

性能对比

```
1  // 测试性能差异
2  async function testPerformance() {
3    console.time("Serial");
4    await loadSerially();
5    console.timeEnd("Serial"); // ~3000ms
6
7    console.time("Parallel");
8    await loadInParallel();
9    console.timeEnd("Parallel"); // ~1000ms
10 }
```

TIP

当多个异步操作互不依赖时，总是使用并行执行以提高性能。

4.4 常见陷阱与解决方案

async/await 虽然简化了异步编程，但也有一些常见的陷阱需要注意。

陷阱 1：忘记 await

```
1  // ❌ 错误：忘记 await
2  async function loadData() {
3    const data = fetchData(); // 返回 Promise，不是实际数据
```

```
4 console.log(data); // Promise { <pending> }
5 }
6
7 // ✅ 正确: 添加 await
8 async function loadData() {
9   const data = await fetchData();
10  console.log(data); // 实际数据
11 }
```

❗ 陷阱 2: 在循环中串行执行

```
1 // ❌ 错误: 串行执行, 性能差 ⚡ JavaScript
2 async function loadUsers(userIds) {
3   const users = [];
4   for (const id of userIds) {
5     const user = await fetchUser(id); // 每次等待
6     users.push(user);
7   }
8   return users;
9 }
10
11 // ✅ 正确: 并行执行
12 async function loadUsers(userIds) {
13   const promises = userIds.map(id => fetchUser(id));
14   return await Promise.all(promises);
15 }
```

性能对比:

```
1 // 假设有 100 个用户, 每个请求 100ms ⚡ JavaScript
2 // 串行: 100 * 100ms = 10s
3 // 并行: max(100ms) = 100ms
```

❗ 陷阱 3: 未处理的错误

```
1 // ❌ 错误: 未捕获的错误 ⚡ JavaScript
2 async function loadData() {
3   const data = await fetchData(); // 如果失败, 错误未被捕获
4   return data;
5 }
6
7 // ✅ 正确: 添加错误处理
8 async function loadData() {
9   try {
10    const data = await fetchData();
11    return data;
12  } catch (error) {
13    console.error(error);
14    return null;
15  }
16 }
```

陷阱 4：顶层 await 的限制

```
1 // ❌ 错误：在非 async 函数中使用 await ⚡ JavaScript
2 function loadData() {
3   const data = await fetchData(); // SyntaxError
4 }
5
6 // ✅ 正确 1：在 async 函数中
7 async function loadData() {
8   const data = await fetchData();
9 }
10
11 // ✅ 正确 2：顶层 await (ES Module)
12 // 只在 ES Module 中支持
13 const data = await fetchData();
```

NOTE

顶层 await 仅在 ES Module 中支持，CommonJS 不支持。

陷阱 5：过度使用 async/await

```
1 // ❌ 不必要：简单的 Promise 链 ⚡ JavaScript
2 async function loadData() {
3   const response = await fetch("/api/data");
4   const data = await response.json();
5   return data;
6 }
7
8 // ✅ 更简洁：直接返回 Promise
9 function loadData() {
10   return fetch("/api/data")
11     .then(response => response.json());
12 }
```

TIP

如果函数只是简单地转发 Promise，不需要使用 async/await。

陷阱 6：错误堆栈丢失

```
1 // ❌ 可能丢失堆栈信息 ⚡ JavaScript
2 async function loadData() {
3   try {
4     await fetchData();
5   } catch (error) {
6     throw new Error("Failed to load"); // 原始堆栈丢失
7   }
8 }
9
10 // ✅ 保留原始错误
11 async function loadData() {
12   try {
13     await fetchData();
```

```

14   } catch (error) {
15     error.message = "Failed to load: " + error.message;
16     throw error; // 保留原始堆栈
17   }
18 }

```

最佳实践总结

实践	说明	示例
总是 await	确保获取实际值	const data = await fetchData()
并行执行	互不依赖的请求并行	Promise.all
错误处理	使用 try-catch	try { await ... } catch {}
避免嵌套	扁平化异步代码	减少 then 链
简洁返回	简单场景直接返回 Promise	return fetch()
保留堆栈	不要创建新错误	修改原错误消息

完整示例

```

1  // 真实的 API 请求封装
2  class ApiService {
3    constructor(baseUrl) {
4      this.baseUrl = baseUrl;
5    }
6
7    async request(endpoint, options = {}) {
8      const url = `${this.baseUrl}${endpoint}`;
9
10     try {
11       const response = await fetch(url, {
12         headers: {
13           "Content-Type": "application/json",
14           ...options.headers
15         },
16         ...options
17       });
18
19       if (!response.ok) {
20         throw new Error(`HTTP ${response.status}: ${response.statusText}`);
21       }
22
23       return await response.json();
24
25     } catch (error) {
26       console.error(`Request failed: ${endpoint}`, error);
27       throw error;
28     }
29   }
30
31   async getUsers() {

```

```
32     return this.request("/users");
33 }
34
35 async getUser(id) {
36     return this.request(`/users/${id}`);
37 }
38
39 async createUser(userData) {
40     return this.request("/users", {
41         method: "POST",
42         body: JSON.stringify(userData)
43     });
44 }
45
46 async loadDashboard(userId) {
47     // 第一步：获取用户
48     const user = await this.getUser(userId);
49
50     // 第二步：并行获取相关数据
51     const [posts, comments, stats] = await Promise.all([
52         this.request(`/users/${userId}/posts`),
53         this.request(`/users/${userId}/comments`),
54         this.request(`/users/${userId}/stats`)
55     ]);
56
57     return {
58         user,
59         posts,
60         comments,
61         stats
62     };
63 }
64 }
65
66 // 使用
67 const api = new ApiService("https://api.example.com");
68
69 async function main() {
70     try {
71         const dashboard = await api.loadDashboard(123);
72         console.log("Dashboard loaded:", dashboard);
73     } catch (error) {
74         console.error("Failed to load dashboard:", error);
75     }
76 }
77
78 main();
```

本节详细介绍了 `async/await` 的基本语法、错误处理、并行执行以及常见陷阱。`async/await` 让异步代码更加清晰易读，是现代 JavaScript 异步编程的首选方式。

5 高级异步模式

在实际开发中，除了基础的 Promise 和 async/await，还需要掌握一些高级的异步编程模式来解决复杂场景。

5.1 回调地狱与解决方案

回调地狱（Callback Hell）是早期 JavaScript 异步编程的主要问题。

什么是回调地狱

```
1 // ❌ 回调地狱：嵌套层级深，难以维护
2 getUser(userId, function(user) {
3   getPosts(user.id, function(posts) {
4     getComments(posts[0].id, function(comments) {
5       getAuthor(comments[0].authorId, function(author) {
6         console.log(author);
7       });
8     });
9   });
10 });
```

问题：

问题	说明
嵌套过深	代码向右延伸，难以阅读
错误处理困难	每层都需要单独处理错误
控制流复杂	难以实现并行、竞争等逻辑
调试困难	堆栈跟踪不清晰

解决方案 1: Promise 链式调用

```
1 // ✅ Promise 链式调用
2 getUser(userId)
3   .then(user => getPosts(user.id))
4   .then(posts => getComments(posts[0].id))
5   .then(comments => getAuthor(comments[0].authorId))
6   .then(author => {
7     console.log(author);
8   })
9   .catch(error => {
10    console.error(error);
11  });
```

解决方案 2: async/await

```
1 // ✅ async/await: 最清晰的写法
2 async function loadAuthorInfo(userId) {
3   try {
4     const user = await getUser(userId);
```

```
5     const posts = await getPosts(user.id);
6     const comments = await getComments(posts[0].id);
7     const author = await getAuthor(comments[0].authorId);
8
9     console.log(author);
10    return author;
11  } catch (error) {
12    console.error(error);
13  }
14 }
```

解决方案 3: 命名函数

```
1  //  提取为命名函数 JavaScript
2  function handleUser(user) {
3    return getPosts(user.id);
4  }
5
6  function handlePosts(posts) {
7    return getComments(posts[0].id);
8  }
9
10 function handleComments(comments) {
11   return getAuthor(comments[0].authorId);
12 }
13
14 getUser(userId)
15   .then(handleUser)
16   .then(handlePosts)
17   .then(handleComments)
18   .then(author => console.log(author))
19   .catch(console.error);
```

5.2 发布-订阅模式

发布-订阅 (Pub/Sub) 模式是一种事件驱动的异步模式，解耦生产者和消费者。

基本实现

```
1  class EventEmitter { JavaScript
2    constructor() {
3      this.events = {};
4    }
5
6    // 订阅事件
7    on(event, callback) {
8      if (!this.events[event]) {
9        this.events[event] = [];
10     }
11     this.events[event].push(callback);
12
13     // 返回取消订阅函数
```

```
14     return () => {
15         this.events[event] = this.events[event].filter(cb => cb !== callback);
16     };
17 }
18
19 // 发布事件
20 emit(event, ...args) {
21     if (this.events[event]) {
22         this.events[event].forEach(callback => {
23             callback(...args);
24         });
25     }
26 }
27
28 // 只订阅一次
29 once(event, callback) {
30     const unsubscribe = this.on(event, (...args) => {
31         callback(...args);
32         unsubscribe(); // 自动取消订阅
33     });
34 }
35
36 // 取消所有订阅
37 off(event) {
38     delete this.events[event];
39 }
40 }
```

使用示例

```
1  const emitter = new EventEmitter();
2
3  // 订阅事件
4  const unsubscribe = emitter.on("data", (data) => {
5      console.log("收到数据:", data);
6  });
7
8  // 发布事件
9  emitter.emit("data", { id: 1, value: "test" });
10 // 输出: 收到数据: { id: 1, value: "test" }
11
12 // 取消订阅
13 unsubscribe();
14 emitter.emit("data", { id: 2, value: "test2" });
15 // 不会输出, 因为已取消订阅
```

 JavaScript

异步事件处理

```
1  class AsyncEventEmitter extends EventEmitter {
2      // 异步发布, 等待所有处理器完成
3      async emitAsync(event, ...args) {
4          if (this.events[event]) {
```

 JavaScript

```
5     const promises = this.events[event].map(callback => {
6       try {
7         return Promise.resolve(callback(...args));
8       } catch (error) {
9         return Promise.reject(error);
10      }
11    });
12
13    return Promise.allSettled(promises);
14  }
15 }
16 }
17
18 // 使用
19 const asyncEmitter = new AsyncEventEmitter();
20
21 asyncEmitter.on("request", async (url) => {
22   const response = await fetch(url);
23   return response.json();
24 });
25
26 asyncEmitter.on("request", async (url) => {
27   console.log("Logging request to:", url);
28 });
29
30 // 发布并等待所有处理器完成
31 const results = await asyncEmitter.emitAsync("request", "https://api.example.com");
32 console.log(results);
33 // [
34 //   { status: "fulfilled", value: {...} },
35 //   { status: "fulfilled", value: undefined }
36 // ]
```

应用场景

场景	说明	示例
事件驱动架构	组件间通信	UI 事件、消息队列
插件系统	扩展点机制	Webpack 插件、Babel 插件
状态管理	状态变化通知	Redux、Vuex
实时通信	WebSocket 消息	聊天应用、实时通知
日志系统	多目标日志	控制台、文件、远程服务器

5.3 生产者-消费者模式

生产者-消费者模式用于处理异步数据流，解耦数据生产和消费。

基于 Promise 的实现

```
1 class TaskQueue {
```

 JavaScript

```
2   constructor(concurrency = 1) {
3     this.queue = [];
4     this.concurrency = concurrency;
5     this.running = 0;
6   }
7
8   // 添加任务
9   add(task) {
10    return new Promise((resolve, reject) => {
11      this.queue.push({
12        task,
13        resolve,
14        reject
15      });
16
17      this.process();
18    });
19  }
20
21  // 处理任务
22  async process() {
23    if (this.running >= this.concurrency || this.queue.length === 0) {
24      return;
25    }
26
27    this.running++;
28    const { task, resolve, reject } = this.queue.shift();
29
30    try {
31      const result = await task();
32      resolve(result);
33    } catch (error) {
34      reject(error);
35    } finally {
36      this.running--;
37      this.process(); // 处理下一个任务
38    }
39  }
40 }
```

使用：

```
1  const queue = new TaskQueue(3); // 最多 3 个并发
2
3  // 添加 10 个任务
4  for (let i = 0; i < 10; i++) {
5    queue.add(() => {
6      console.log(`Task ${i} started`);
7      return new Promise(resolve => {
8        setTimeout(() => {
9          console.log(`Task ${i} completed`);
10         resolve(i);
11       }, 1000);
12     });
13   }
```

 JavaScript

```
12     });
13   }).then(result => {
14     console.log(`Result: ${result}`);
15   });
16 }
17
18 // 输出:
19 // Task 0 started
20 // Task 1 started
21 // Task 2 started
22 // (1秒后)
23 // Task 0 completed
24 // Result: 0
25 // Task 3 started
26 // ...
```

基于生成器的实现

```
1  function* producer() {
2    let i = 0;
3    while (true) {
4      yield i++;
5    }
6  }
7
8  function* consumer(producer) {
9    while (true) {
10     const value = producer.next().value;
11     console.log("Consumed:", value);
12     yield value;
13   }
14 }
15
16 // 使用
17 const prod = producer();
18 const cons = consumer(prod);
19
20 cons.next(); // Consumed: 0
21 cons.next(); // Consumed: 1
22 cons.next(); // Consumed: 2
```

 JavaScript

基于 AsyncIterator 的实现

```
1  class AsyncQueue {
2    constructor() {
3      this.items = [];
4      this.resolvers = [];
5    }
6
7    // 生产者: 添加数据
8    push(item) {
9      if (this.resolvers.length > 0) {
```

 JavaScript

```
10     const resolve = this.resolvers.shift();
11     resolve({ value: item, done: false });
12   } else {
13     this.items.push(item);
14   }
15 }
16
17 // 消费者：获取数据
18 pop() {
19   if (this.items.length > 0) {
20     return Promise.resolve({
21       value: this.items.shift(),
22       done: false
23     });
24   }
25
26   return new Promise(resolve => {
27     this.resolvers.push(resolve);
28   });
29 }
30
31 // 实现 AsyncIterator
32 [Symbol.asyncIterator]() {
33   return {
34     next: () => this.pop()
35   };
36 }
37 }
```

使用：

```
1  const queue = new AsyncQueue();
2
3  // 消费者：异步迭代
4  (async () => {
5    for await (const item of queue) {
6      console.log("Processed:", item);
7    }
8  })();
9
10 // 生产者：添加数据
11 setTimeout(() => queue.push(1), 1000);
12 setTimeout(() => queue.push(2), 2000);
13 setTimeout(() => queue.push(3), 3000);
14
15 // 输出：
16 // (1秒后) Processed: 1
17 // (2秒后) Processed: 2
18 // (3秒后) Processed: 3
```

JavaScript

应用场景

场景	说明	示例
----	----	----

任务队列	限制并发数	图片上传、API 请求
数据流处理	背压控制	文件读写、网络流
消息队列	异步消息处理	RabbitMQ、Kafka
批处理	批量处理数据	数据库批量插入
工作线程池	CPU 密集型任务	图像处理、计算

5.4 异步迭代器与生成器

异步迭代器和生成器提供了处理异步数据流的优雅方式。

生成器基础

生成器函数可以暂停和恢复执行。

```
1 function* numberGenerator() {  
2   yield 1;  
3   yield 2;  
4   yield 3;  
5 }  
6  
7 const gen = numberGenerator();  
8  
9 console.log(gen.next()); // { value: 1, done: false }  
10 console.log(gen.next()); // { value: 2, done: false }  
11 console.log(gen.next()); // { value: 3, done: false }  
12 console.log(gen.next()); // { value: undefined, done: true }
```

异步生成器

异步生成器可以 yield Promise。

```
1 async function* asyncNumberGenerator() {  
2   yield await Promise.resolve(1);  
3   yield await Promise.resolve(2);  
4   yield await Promise.resolve(3);  
5 }  
6  
7 // 使用 for-await-of  
8 (async () => {  
9   for await (const num of asyncNumberGenerator()) {  
10    console.log(num); // 1, 2, 3  
11   }  
12 })();
```

实战：分页数据加载

```
1 async function* fetchPages(baseUrl) {  
2   let page = 1;  
3   let hasMore = true;  
4  
5   while (hasMore) {
```



```
6     const response = await fetch(`${baseUrl}?page=${page}`);
7     const data = await response.json();
8
9     for (const item of data.items) {
10         yield item;
11     }
12
13     hasMore = data.hasMore;
14     page++;
15 }
16 }
17
18 // 使用：像同步代码一样处理异步数据流
19 (async () => {
20     for await (const item of fetchPages("https://api.example.com/items")) {
21         console.log(item);
22         // 可以在这里处理每个 item
23         // 不需要关心分页逻辑
24     }
25 })();
```

实战：无限滚动

```
1  async function* infiniteScroll(fetchFn) { ⚡ JavaScript
2      let cursor = null;
3
4      while (true) {
5          const { items, nextCursor } = await fetchFn(cursor);
6
7          for (const item of items) {
8              yield item;
9          }
10
11          if (!nextCursor) {
12              break;
13          }
14
15          cursor = nextCursor;
16      }
17 }
18
19 // 使用
20 const fetchPosts = async (cursor) => {
21     const url = cursor
22       ? `/api/posts?cursor=${cursor}`
23       : "/api/posts";
24
25     const response = await fetch(url);
26     return response.json();
27 };
28
29 (async () => {
30     let count = 0;
```

```
31   for await (const post of infiniteScroll(fetchPosts)) {
32     console.log(post.title);
33     count++;
34
35     // 加载 100 条后停止
36     if (count >= 100) {
37       break;
38     }
39   }
40 }());
```

异步迭代器协议

手动实现异步迭代器。

```
1  class AsyncRange { ⚡ JavaScript
2    constructor(start, end) {
3      this.start = start;
4      this.end = end;
5    }
6
7    [Symbol.asyncIterator]() {
8      let current = this.start;
9
10     return {
11       async next() {
12         // 模拟异步操作
13         await new Promise(resolve => setTimeout(resolve, 100));
14
15         if (current < this.end) {
16           return { value: current++, done: false };
17         } else {
18           return { done: true };
19         }
20       }
21     };
22   }
23 }
24
25 // 使用
26 (async () => {
27   for await (const num of new AsyncRange(0, 5)) {
28     console.log(num); // 0, 1, 2, 3, 4 (每个间隔 100ms)
29   }
30 })();
```

组合异步迭代器

```
1  // 过滤 ⚡ JavaScript
2  async function* filter(asyncIterable, predicate) {
3    for await (const item of asyncIterable) {
4      if (predicate(item)) {
5        yield item;
```

```
6     }
7   }
8 }
9
10 // 映射
11 async function* map(asyncIterable, transform) {
12   for await (const item of asyncIterable) {
13     yield transform(item);
14   }
15 }
16
17 // 限制数量
18 async function* take(asyncIterable, count) {
19   let taken = 0;
20   for await (const item of asyncIterable) {
21     if (taken >= count) break;
22     yield item;
23     taken++;
24   }
25 }
26
27 // 组合使用
28 (async () => {
29   const numbers = new AsyncRange(0, 100);
30
31   const result = await Array.fromAsync(
32     take(
33       filter(
34         map(numbers, n => n * 2),
35         n => n % 3 === 0
36       ),
37       5
38     )
39   );
40
41   console.log(result); // [0, 6, 12, 18, 24]
42 })();
```

NOTE

`Array.fromAsync()` 是 ES2023 新增的方法，用于将异步迭代器转换为数组。

应用场景

场景	说明	示例
分页数据	逐页加载	无限滚动、懒加载
数据流处理	流式处理大数据	文件读取、网络流
实时数据	WebSocket 消息流	股票行情、聊天消息
数据库查询	游标遍历	大量数据查询
管道处理	数据转换管道	ETL、数据处理

本节介绍了高级异步模式，包括回调地狱的解决方案、发布-订阅模式、生产者-消费者模式以及异步迭代器与生成器。这些模式能够帮助你解决复杂的异步编程场景，写出更优雅、更可维护的代码。

6 定时器与异步调度

6.1 setTimeout / setInterval

6.2 requestAnimationFrame

6.3 setImmediate (Node.js)

6.4 process.nextTick (Node.js)

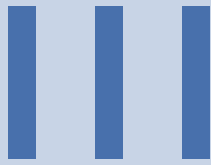
7 实战示例

7.1 API 请求封装

7.2 并发控制

7.3 重试机制

7.4 超时控制



TypeScript 类型系统与高级特性

IV

标准库与常用 API



浏览器端 JavaScript

VI

Node.js

VII

JavaScript 工程化

VIII

JavaScript 底层原理